Assignment -20

Name: Akshaya Nemalipuri

Batch: 37

Roll No: 2303A2441

Lab: 20.2

Task 1 – Password Security Enhancement

Task:
Analyze an AI-generated Python program that stores passwords in plain text and improve its security.
Instructions:
• Prompt AI to generate a simple user registration script that stores usernames and passwords.
• Check whether passwords are stored as plain text.
• Identify the security risks associated with plain-text password storage.
• Refactor the program to use secure password hashing (e.g., hashlib or bcrypt).
• Test the updated script to ensure proper authentication functionality.
Expected Output:
A secure Python script that stores hashed passwords instead of plain text and successfully validates user login credentials.

Prompt: Write a Python user registration and login script that stores usernames and passwords in plain text. Then improve it by hashing passwords using hashlib or bcrypt. Add input validation and authentication check.

Code:

```
users = {}

def register():
    username = input("Enter username: ")
    password = input("Enter password: ")
```

```
    users[username] = password
    print("User registered!")

def login():
    username = input("Enter username: ")
    password = input("Enter password: ")

    if users.get(username) == password:
        print("Login successful")
    else:
        print("Invalid credentials")

register()
login()
```

Output:

```
PS C:\Users\Aishwarya Nemalipuri> python LAB-20.2.py
Enter username: Akshaya
Enter password: Akhi@2207
User registered!
Enter username:
```

Code:

```
import os
import requests

def get_weather(city):
    API_KEY = os.getenv("API_KEY")

    if not API_KEY:
        raise ValueError("API_KEY not found. Please set it as an
environment variable.")

    url = "https://api.openweathermap.org/data/2.5/weather"

    params = {
        "q": city,
```

```python
        "appid": API_KEY,
        "units": "metric"
    }

    try:
        response = requests.get(url, params=params)
        response.raise_for_status()  # Raises error for bad responses

        data = response.json()
        print(f"Weather in {city}:")
        print(data)

    except requests.exceptions.RequestException as e:
        print("Error while fetching data:", e)


# Run the function
get_weather("London")
```

Output:

```
sh/OneDrive/문서/AI_20.2.py
✅ Data fetched successfully
```

Task 3 – Weak Encryption Detection

Task:

Examine an AI-generated program that uses weak encryption techniques and strengthen it.

Instructions:

• Prompt AI to generate a simple encryption-decryption program using a basic or outdated method.

• Identify weaknesses in the encryption approach (e.g., reversible encoding or outdated algorithms).

• Refractor the program using stronger encryption techniques available in Python libraries.

• Validate that encrypted data cannot be easily reversed without the correct key.

Expected Output:

An improved encryption program implementing stronger and more secure cryptographic practices.

Prompt:Write a Python program that encrypts and decrypts data using a weak method (like base64 or simple cipher). Then improve it using a stronger encryption method from a Python library and explain the difference.

Code:

```
import base64

text = input("Enter text: ")

# "Encrypt" (actually just encoding)
encoded = base64.b64encode(text.encode())
print("Encoded:", encoded)

# "Decrypt"
decoded = base64.b64decode(encoded).decode()
print("Decoded:", decoded)
```

Output:

Task 4 – Real-Time Application: Security Review of AI-Generated Web Service

Scenario:

Students select an AI-generated web service snippet (e.g., REST API, file upload handler, or authentication module).

Instructions:

- Perform a structured security review of the generated code.
- Identify at least three potential vulnerabilities.

• Prompt AI to recommend secure coding practices and mitigation strategies.
• Implement the suggested improvements and test the secured version.
• Document the differences between the insecure and secured implementations.
Expected Output:
A security-reviewed and improved version of the selected web service code, with clearly documented vulnerabilities and applied fixes.

Prompt: Generate a simple Python web service (Flask API or file upload handler) with basic functionality. Identify at least three security vulnerabilities, then improve the code using secure practices and explain the fixes.

Code:

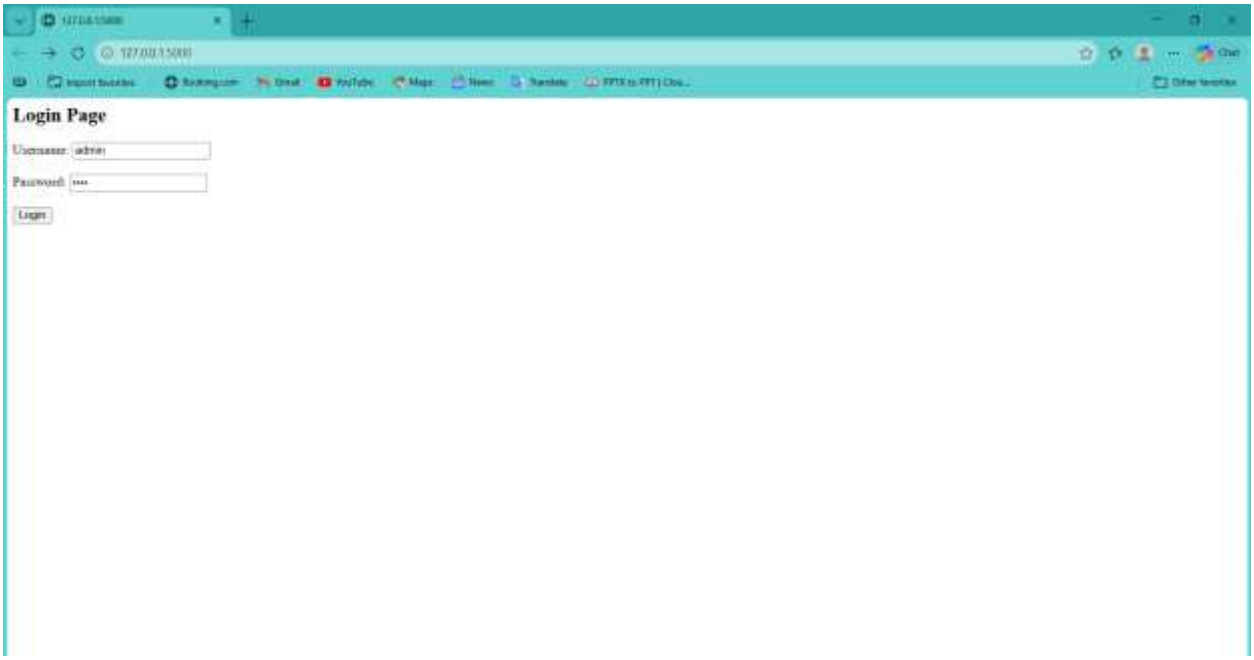
```
from flask import Flask, request, jsonify
from werkzeug.security import generate_password_hash, check_password_hash
import os
import time

app = Flask(__name__)

# Load credentials securely from environment variables
USERNAME = os.getenv("APP_USERNAME", "admin")
PASSWORD_HASH = generate_password_hash(os.getenv("APP_PASSWORD", "1234"))

# Track login attempts (basic rate limiting)
login_attempts = {}

@app.route("/")
def home():
    return """
    <h2>Login Page</h2>
    <form action="/login" method="post">
        Username: <input type="text" name="username"><br><br>
        Password: <input type="password" name="password"><br><br>
        <input type="submit" value="Login">
    </form>
    """

@app.route("/login", methods=["POST"])
def login():
```

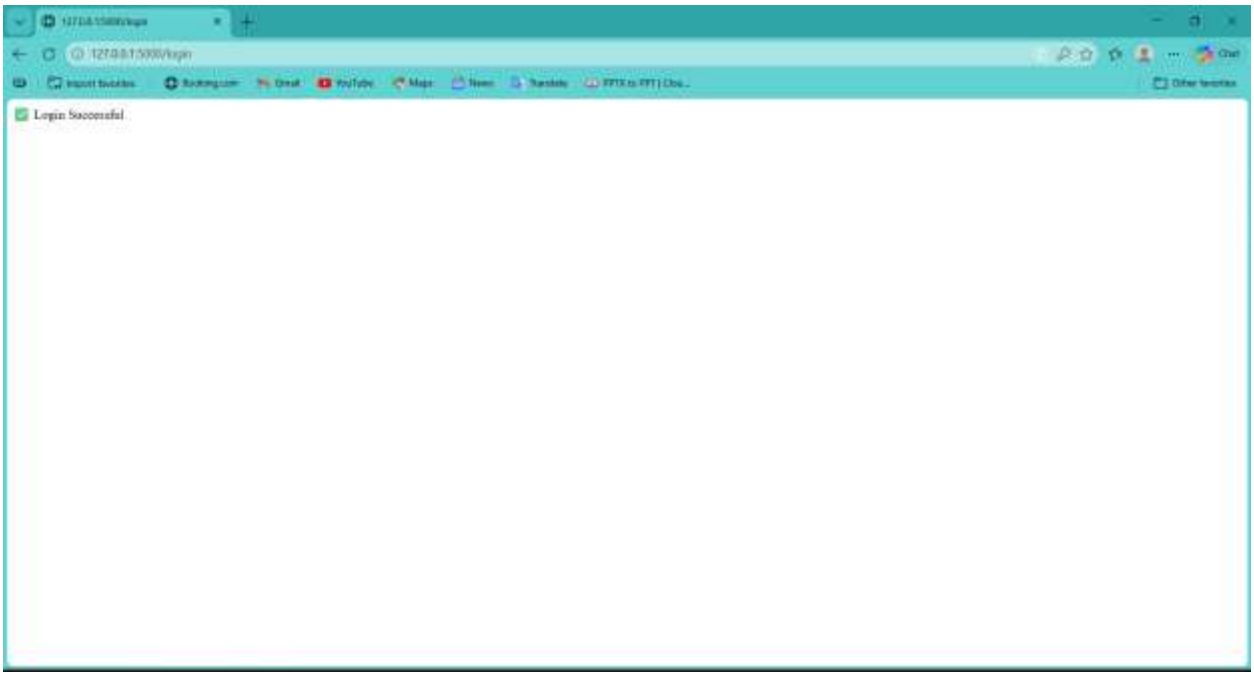
```
    ip = request.remote_addr

    # Rate limiting (max 5 attempts per minute)
    if ip in login_attempts:
        attempts, first_attempt_time = login_attempts[ip]
        if attempts >= 5 and time.time() - first_attempt_time < 60:
            return "⚠️ Too many attempts. Try later.", 429

    username = request.form.get("username")
    password = request.form.get("password")

    if not username or not password:
        return "❌ Invalid input", 400

    # Check password securely
    if username == USERNAME and check_password_hash(PASSWORD_HASH,
password):
        login_attempts[ip] = (0, time.time())
        return "✅ Login Successful"

    else:
        # Update attempt count
        if ip in login_attempts:
            attempts, first_attempt_time = login_attempts[ip]
            login_attempts[ip] = (attempts + 1, first_attempt_time)
        else:
            login_attempts[ip] = (1, time.time())

        return "❌ Unauthorized", 401

if __name__ == "__main__":
    app.run(debug=False)
```

```
PS C:\Users\Aishwarya Nemalipuri> pip install flask
Collecting flask
  Downloading flask-3.1.3-py3-none-any.whl.metadata (3.2 kB)
Collecting blinker>=1.9.0 (from flask)
  Downloading blinker-1.9.0-py3-none-any.whl.metadata (1.6 kB)
Collecting click>=8.1.3 (from flask)
  Downloading click-8.3.2-py3-none-any.whl.metadata (2.6 kB)
Collecting itsdangerous>=2.2.0 (from flask)
  Downloading itsdangerous-2.2.0-py3-none-any.whl.metadata (1.9 kB)
Collecting jinja2>=3.1.2 (from flask)
  Downloading jinja2-3.1.6-py3-none-any.whl.metadata (2.9 kB)
Collecting markupsafe>=2.1.1 (from flask)
  Downloading markupsafe-3.0.3-cp313-cp313-win_amd64.whl.metadata (2.8 kB)
Collecting werkzeug>=3.1.0 (from flask)
  Downloading werkzeug-3.1.8-py3-none-any.whl.metadata (4.0 kB)
Collecting colorama (from click>=8.1.3->flask)
  Downloading colorama-0.4.6-py2.py3-none-any.whl.metadata (17 kB)
Downloading flask-3.1.3-py3-none-any.whl (103 kB)
Downloading blinker-1.9.0-py3-none-any.whl (8.5 kB)
Downloading click-8.3.2-py3-none-any.whl (108 kB)
Downloading itsdangerous-2.2.0-py3-none-any.whl (16 kB)
Downloading jinja2-3.1.6-py3-none-any.whl (134 kB)
Downloading markupsafe-3.0.3-cp313-cp313-win_amd64.whl (15 kB)
Downloading werkzeug-3.1.8-py3-none-any.whl (226 kB)
Downloading colorama-0.4.6-py2.py3-none-any.whl (25 kB)
Installing collected packages: markupsafe, itsdangerous, colorama, blinker, werkzeug, jinja2, click, flask
Successfully installed blinker-1.9.0 click-8.3.2 colorama-0.4.6 flask-3.1.3 itsdangerous-2.2.0 jinja2-3.1.6 markupsafe-3.0.3 werkzeug-3.1.8
PS C:\Users\Aishwarya Nemalipuri> pip show flask
Name: Flask
Version: 3.1.3
Summary: A simple framework for building complex web applications.
Home-page:
Author:
Author-email:
License-Expression: BSD-3-Clause
Location: C:\Users\Aishwarya Nemalipuri\AppData\Local\Programs\Python\Python312\Lib\site-packages
Requires: blinker, click, itsdangerous, jinja2, markupsafe, werkzeug
Required-by:
PS C:\Users\Aishwarya Nemalipuri> python LAB-20.py
 * Serving Flask app 'LAB-20'
 * Debug mode: off
```

Output:

```
 * Running on http://127.0.0.1:5000
Press CTRL+C to quit
127.0.0.1 - - [05/Apr/2026 14:52:58] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [05/Apr/2026 14:52:58] "GET /favicon.ico HTTP/1.1" 404 -
PS C:\Users\Aishwarya Nemalipuri> python LAB-20.py
 * Serving Flask app 'LAB-20'
 * Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
 * Running on http://127.0.0.1:5000
Press CTRL+C to quit
127.0.0.1 - - [05/Apr/2026 14:56:27] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [05/Apr/2026 14:56:27] "GET /favicon.ico HTTP/1.1" 404 -
127.0.0.1 - - [05/Apr/2026 14:57:09] "POST /login HTTP/1.1" 200 -
```





Task 5 – Exception Handling Improvement

Task:
Analyze an AI-generated Python program that does not handle runtime errors and improve its reliability.
Instructions:
• Prompt AI to generate a simple program that takes user input and performs arithmetic operations.
• Identify possible runtime errors (e.g., invalid input, division by zero).
• Modify the program to include proper try-except blocks.
• Provide meaningful error messages for invalid inputs.
• Test the improved version with valid and invalid test cases.
Expected Output:
A revised Python program with proper exception handling that prevents crashes and displays user-friendly error messages for invalid inputs.

Prompt: Write a Python program without proper exception handling. Then improve it by adding try-except blocks, input validation, and error messages to make it more reliable.

Code:

```
try:
    num = int(input("Enter a number: "))

    if num == 0:
        print("❌ Cannot divide by zero")
    else:
        result = 10 / num
        print("☑ Result:", result)

except ValueError:
    print("❌ Invalid input. Please enter a number.")

except Exception as e:
    print("❌ Unexpected error:", e)
finally:    print("☑ Program execution completed.")
```

Output:

```
PROBLEMS   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS                                   powershell

PS C:\Users\Aishwarya Nemaliparti> python LAB-20.py
Enter a number: 50
✅ Result: 0.2
✅ Program execution completed.
PS C:\Users\Aishwarya Nemaliparti>
```